

Akillı Mum

Visit for Perfect Assets: <https://www.akillimum.net> Please contact me on: akillimum@gmail.com

Follow on: <https://twitter.com/ArVrUnity>

<https://www.youtube.com/@akillimum>

Skype: hayletbisi

Discord: akillimum

Please review the packages even if you do not like them. And please send me an email first for the errors :) I will solve it for sure ;)

URP Reflective Shaders 4.x

updated on - May 6, 2024



IMPORTANT: I am re-writing the reflective shader package for Unity 2020.2 and later with cleaner and faster code (also solves the lag problem on VR headsets - thanks to **SJAM** for his help). Also using the **shader graph** for the shaders. It also enables you to use mirrored textures on your **custom shaders** easily!

So I am writing a **"MirrorManager"** script to manage the new reflections. If you are using a smaller version than Unity 2020.2 or non-VR; you can still use previous docs and scripts. Please refer to old doc here:

<https://docs.google.com/document/d/1We78qz0Fa54B3fepdrEbr2oAG4SHIA0TTA3qsLBSiHM/edit#>

I will not remove the old "CameraShade" scripts from the package. So as i said, you can use them too for non-VR apps (until i finish the whole implementation with new "MirrorManager").

Simple Setup Video:

[How to create a perfect mirror - Unity3d - URP](#)

So what this version includes:

This version is a re-written version of the current reflective scripts. It solves the VR lag problem and runs faster. But includes **full mirror and reflective surface shader graph samples and handwritten complex shader like the older version to mimic URP Lit shader, so complex shader can handle all previous samples (depth, alpha mask etc).** I will add other samples as soon as possible!

If you do not target VR or using Unity below 2020.2:

I will implement all script-shader properties soon for this version (WIP), but until that time you can use old scripts and shaders as I mentioned before. Here is the previous docs:

<https://docs.google.com/document/d/1We78qz0Fa54B3fepdrEbr2oAG4SHIA0TTA3qsLBSiHM/edit#>

Extra:

Package also includes "Rotatable Box Projection Probe Shader". Please see it's doc [HERE](#)

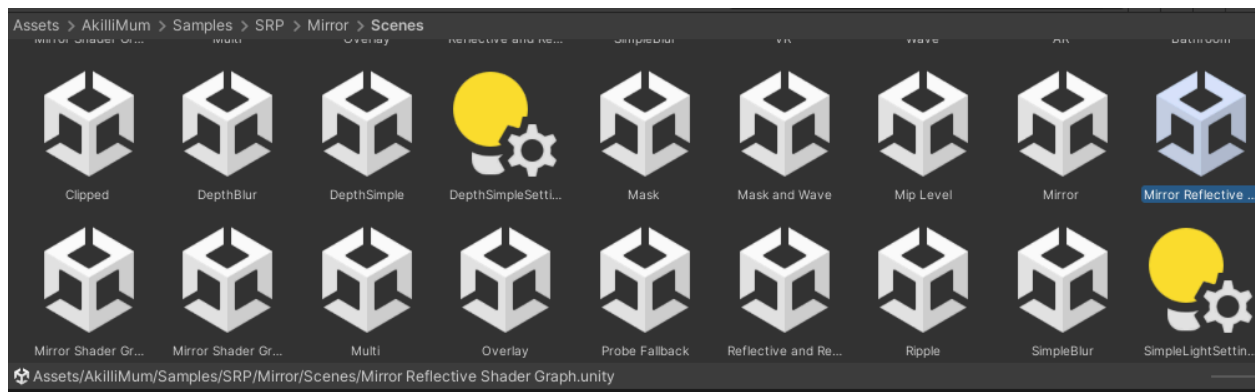
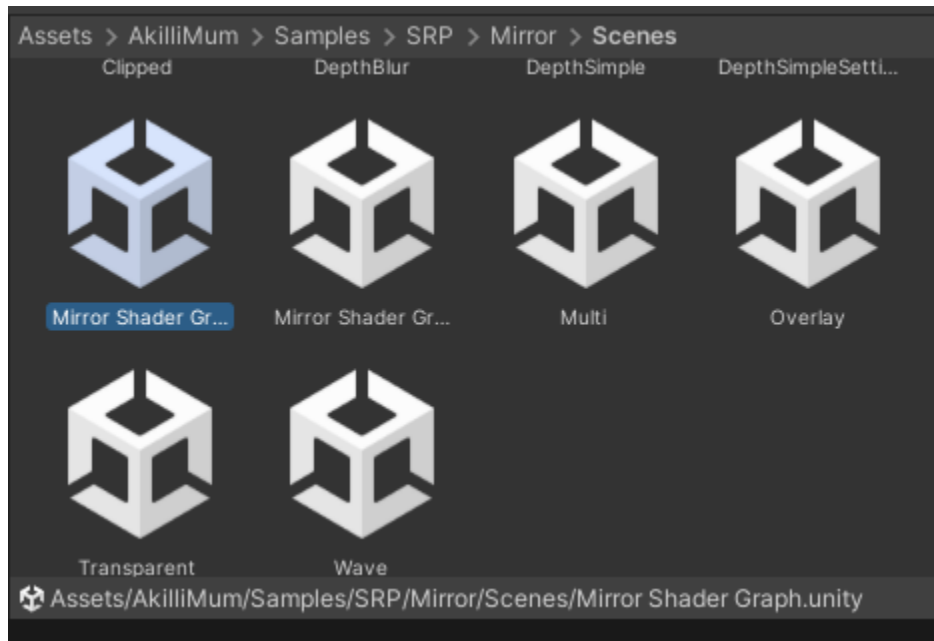
Known Bugs and Issues:

* URP new renderer feature SSAO (Unity's) breaks the reflection

Still investigating if related to my code etc. but SSAO creates glitch dark areas on reflection with SSAO enabled as a renderer feature.

OpenGL Usage

OpenGL may have some limitations (webgl or opengl targeted android or iOS or VR devices like Quest) for the “PBR” shaders. So we wrote some shaderGraphs. Try to use them for those projects: You can see the samples and which shader to use on this samples:



And for VR for openGL please look into these samples:



Limitations:

1. VR works for:

Rift, Rift-S->Single Pass Instanced mode only

Quest, Quest2 standalone->Multi View mode only

Quest, Quest2 with Oculus-Link->Single Pass Instance mode only

Tested on XR-Plugin->Oculus Provider

Tested on XR-Plugin->OpenXR Provider-Oculus Runtime

Users are telling other VR devices are working fine too (like index, HTC); but i can not guarantee because i do not have any of those devices!

2. VR can not draw skyboxes. But I wrote custom shaders to mimic the skybox and to be reflected correctly on the mirror. (Standard Skybox, VR-Stereo Cubemap Skybox and VR-Stereo Equirectangular Panorama Skybox are implemented). If you are using some other skybox solution and can not get it to work please write to me :) Please look under the sample scenes VR folder for a sample fake-sky.

Please see below video to understand how to use (setup) a fake skybox on VR project (It just uses a big cube or sphere to create an illusion of a fake skybox object covers the entire scene):

<https://youtu.be/Ft3q1qmrIOI?t=251>

3. VR implementation will not support recursive (mirror in mirror).

4. Edit: Below problem seems it is a URP bug, please vote it to be solved!

<https://issuetracker.unity3d.com/issues/urp-universalrenderpipeline-dot-rendersinglecamera-method-doesnt-render-ui-components-to-render-texture>

UI can not be reflected on mirror objects. Unity seems to toggle off them on render texture drawings. Please vote for the above issue to be resolved soon.

If the package does not update correctly?

Package may not update correctly on 2020.2.x and later, so please follow below instructions for upgraded projects:

1. Backup :)
2. Delete AkilliMum folder and close the project
3. Goto your asset download folder and delete all (or u can just delete mine named as "Akli Mum"). Folders are here:

macOS: ~/Library/Unity/Asset Store

○ and

○ ~/Library/Unity/Asset Store-5.x

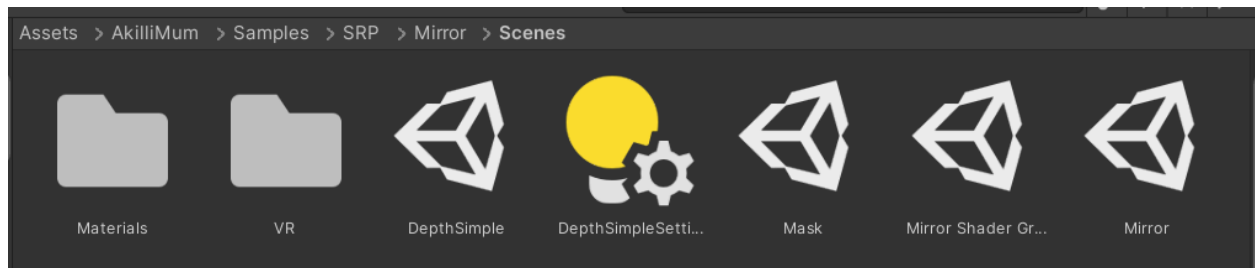
Windows: C:\Users\{accountName}\AppData\Roaming\Unity\Asset Store

○ and

○ C:\Users\{accountName}\AppData\Roaming\Unity\Asset Store-5.x

4. Open the project, download and import again from the package manager :)

Samples:



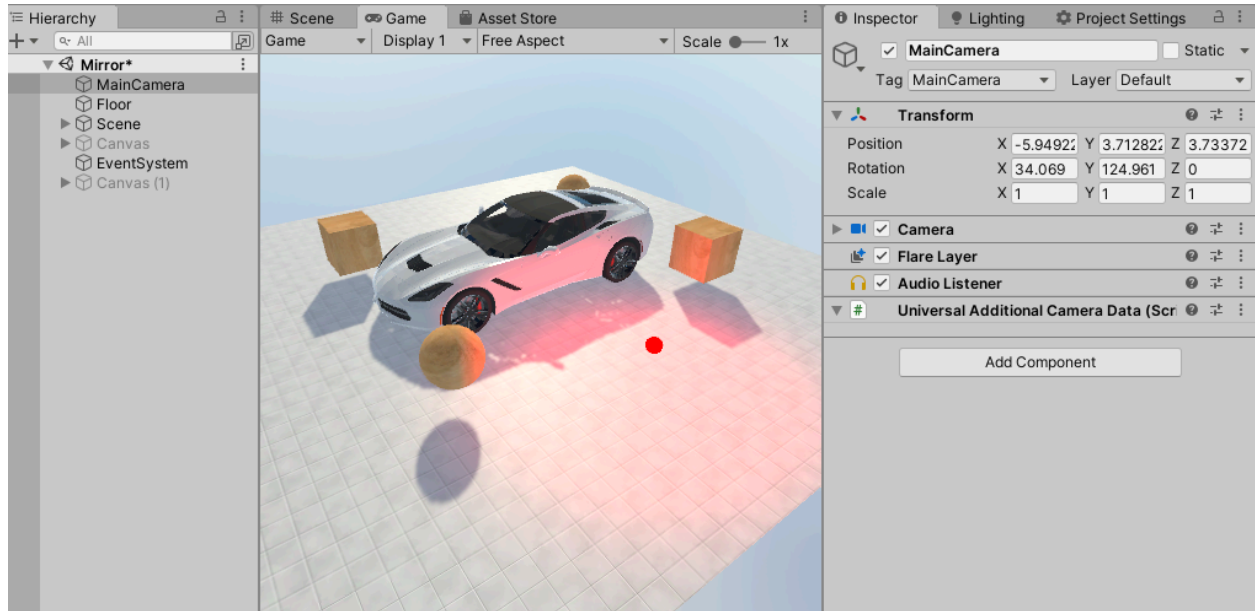
Samples sit on the path in the above picture. Samples ending with “Shader Graph” use a shader graph while others use our new “Complex” shader :)

You can find the VR samples under the “VR” folder.

Setup:

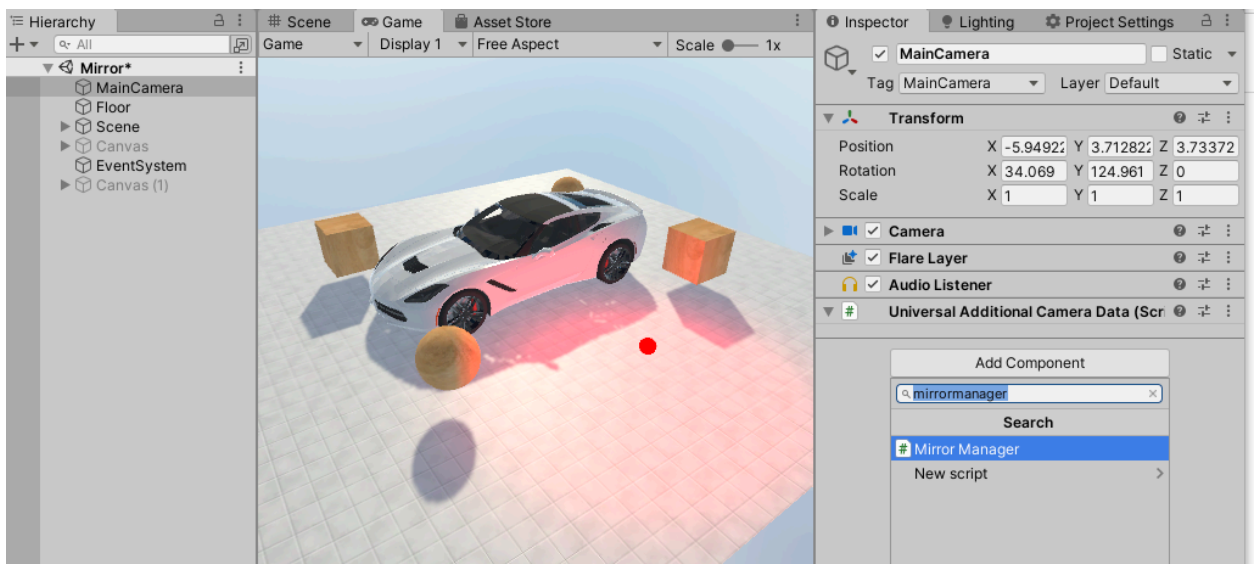
Please see this video for latest setup: [Unity3d - URP - AR VR Ready - Mirror Shaders - Create Simple Mirror - YouTube](#)

Setup is pretty simple. Here is a simple scene with some objects and a floor:

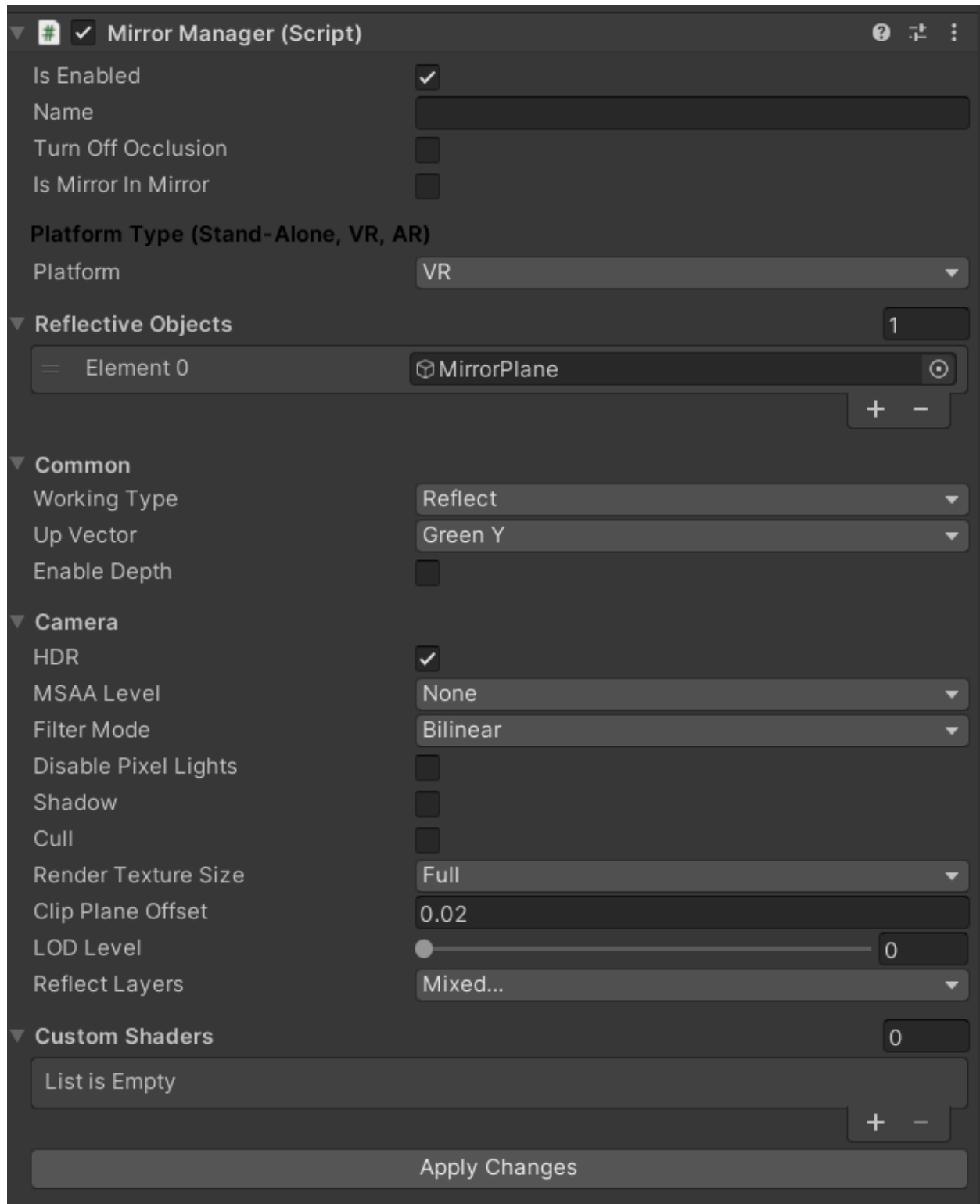


For this sample, I want to make the floor a full mirror!

Let's add our script to the main camera. Click the “Add Component” button when the camera is selected on “Inspector” window and search for “MirrorManager”:

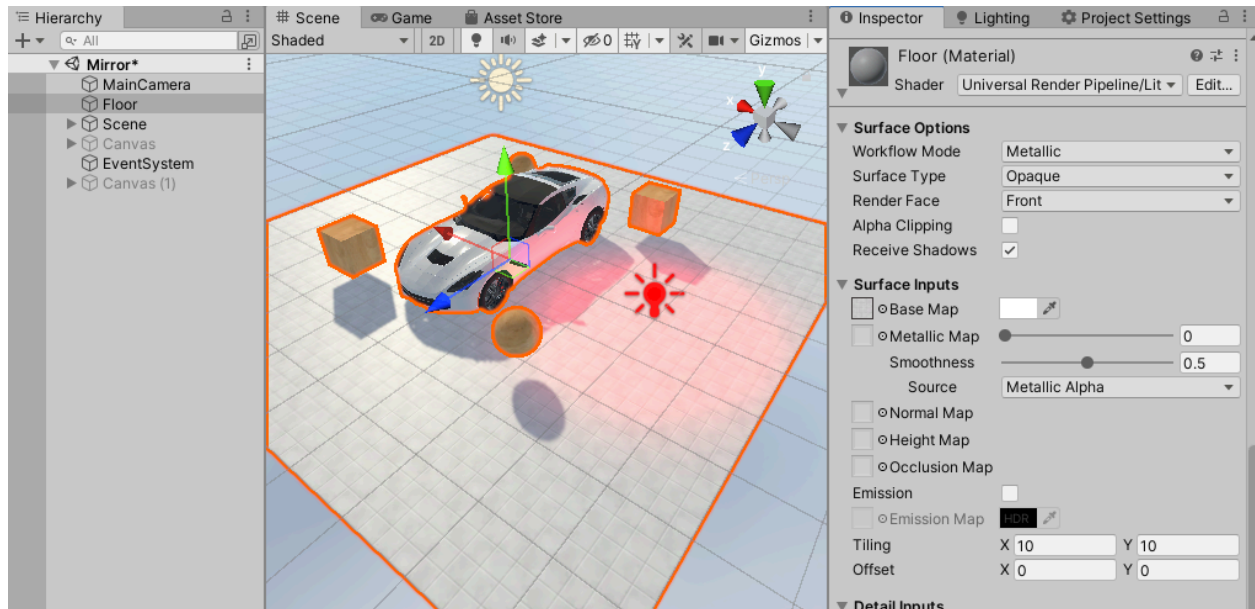


So our script will be added there:

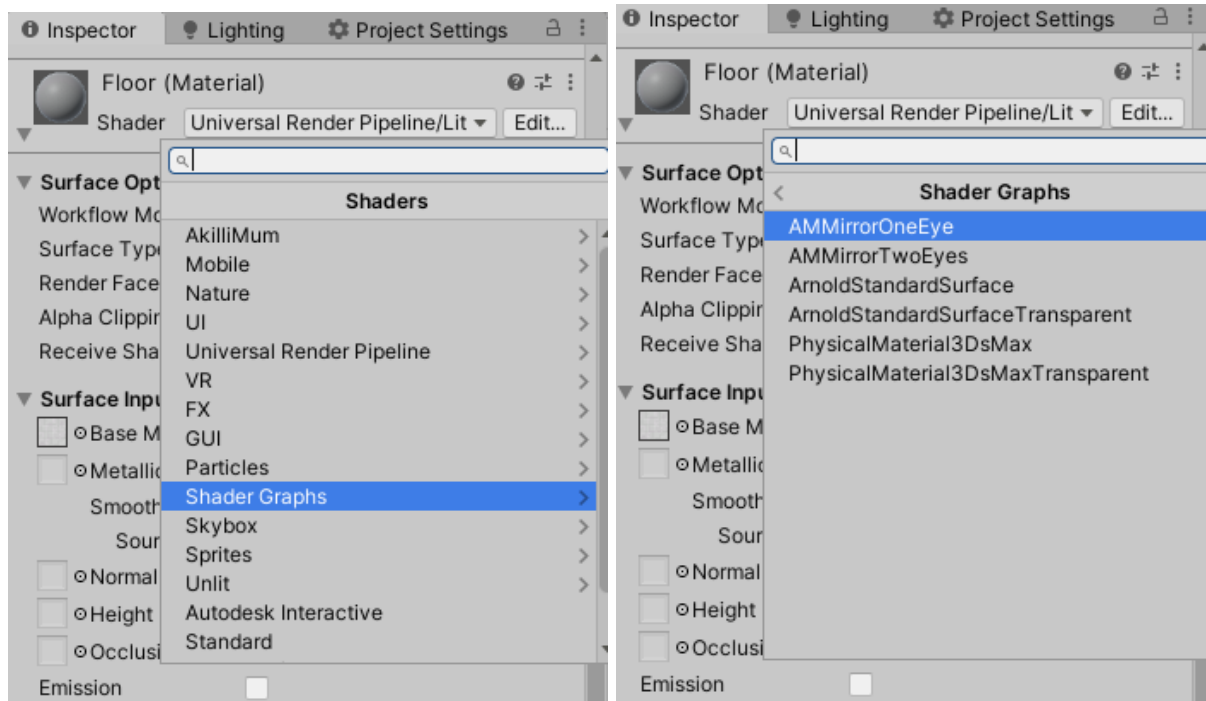


We will come to the properties later :)

Lets select our mirror (floor in this case) and set it up. First select the floor in “scene view” and change the its material’s shader:

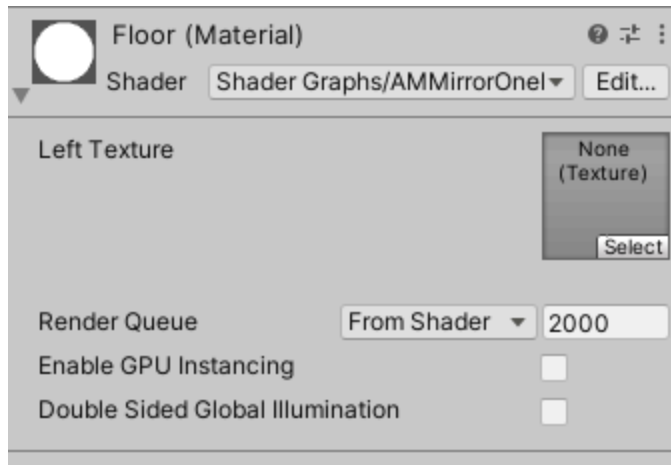


Here you can see my floor uses the standard lit shader of the URP. Just click the “Shader” dropdown and set it to:

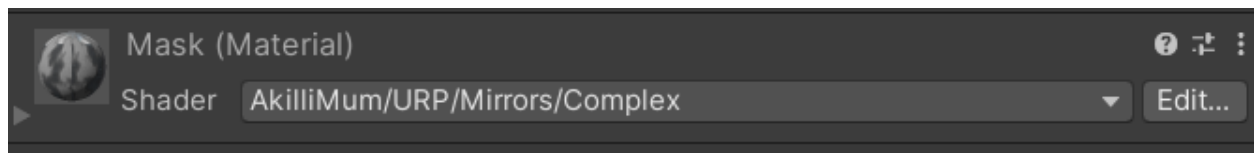


Shader Graphs/AMMirrorOneEye (yes TwoEyes version is for VR :))

So it becomes very simple material:

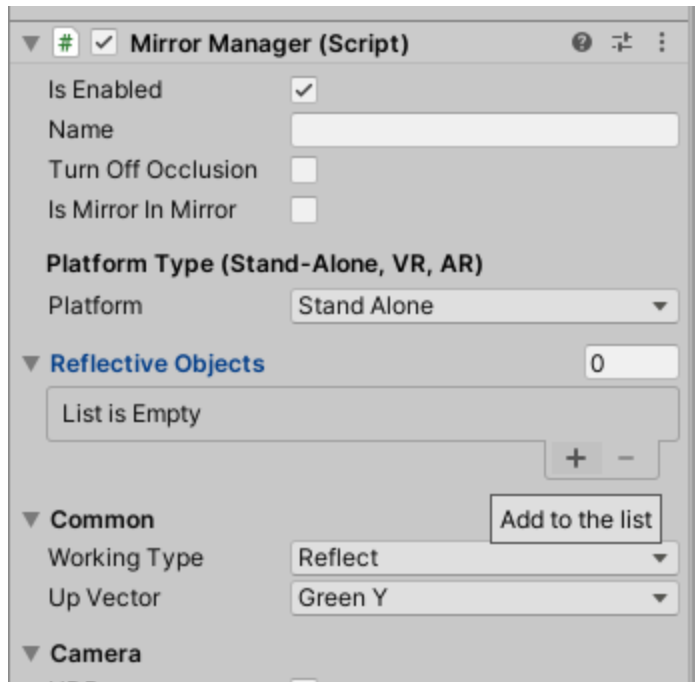


Warning: I have implemented an old handwritten shader to the latest version. So you can use it for complex (PBR) needs. It sits on

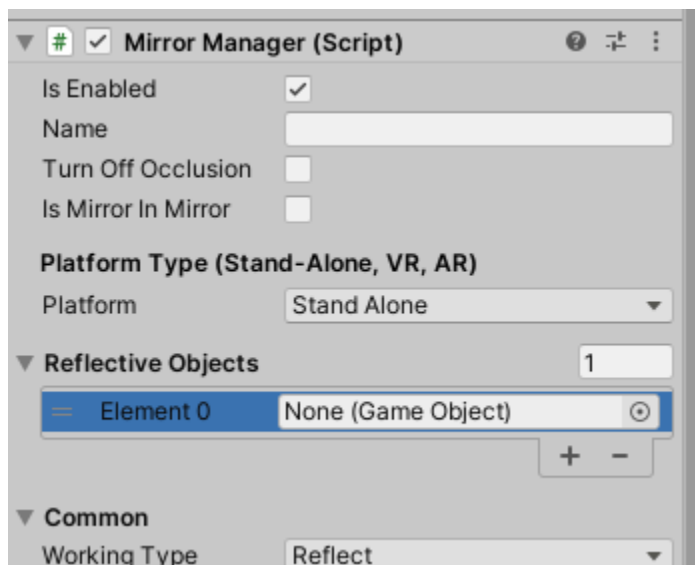


And works for both standalone and VR too. So you do not have to switch it on VR to any shader:) Please look into the samples for usage of it.

And let's back to our script "MirrorManager". We have to attach the mirror object (floor for this case) to the manager. Just go to the "Reflective Objects" list property on the script, open it and add an item:



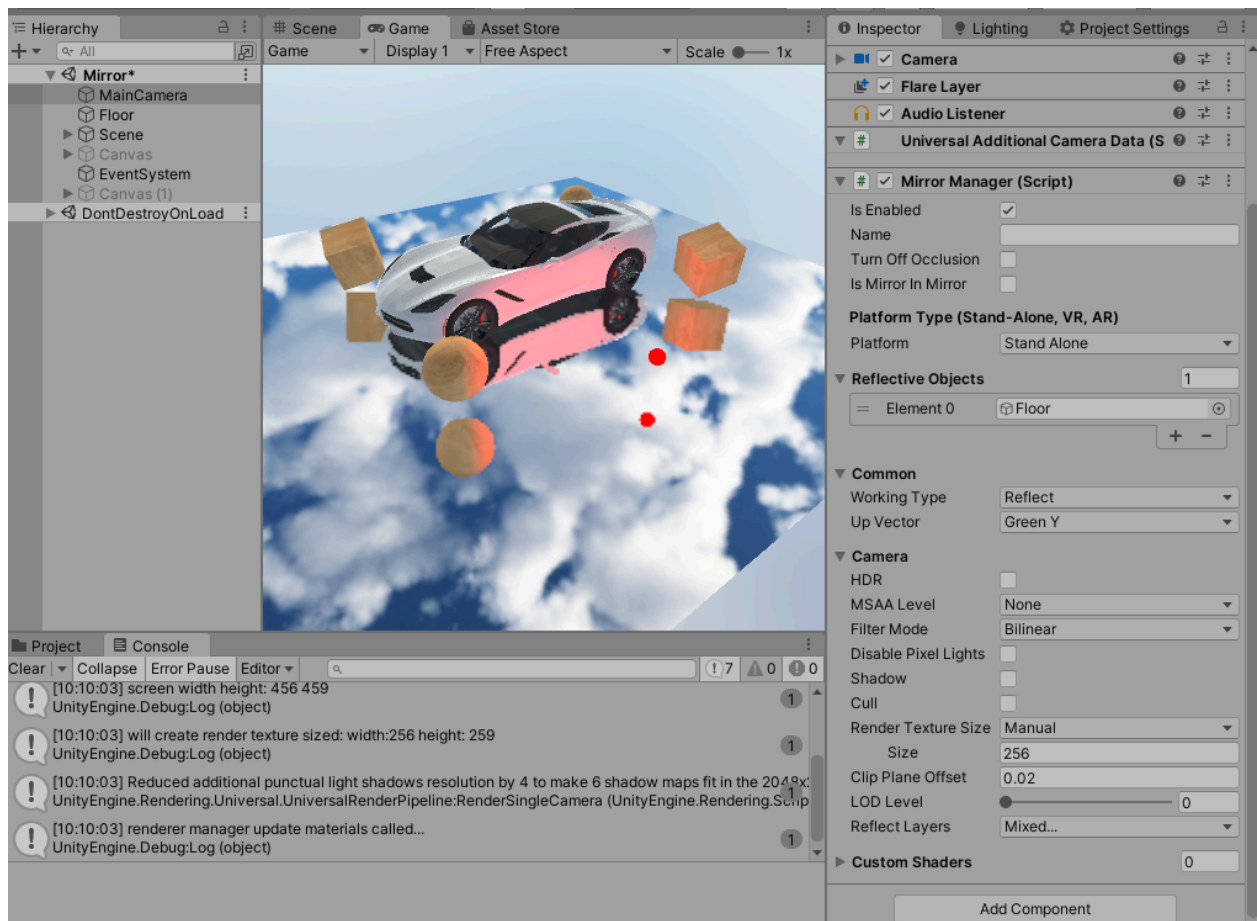
When you hit the “+” sign it will add an empty item to the list:



Now hold the mouse down (not release) on floor object on “Hierarchy” window and drag and drop here:

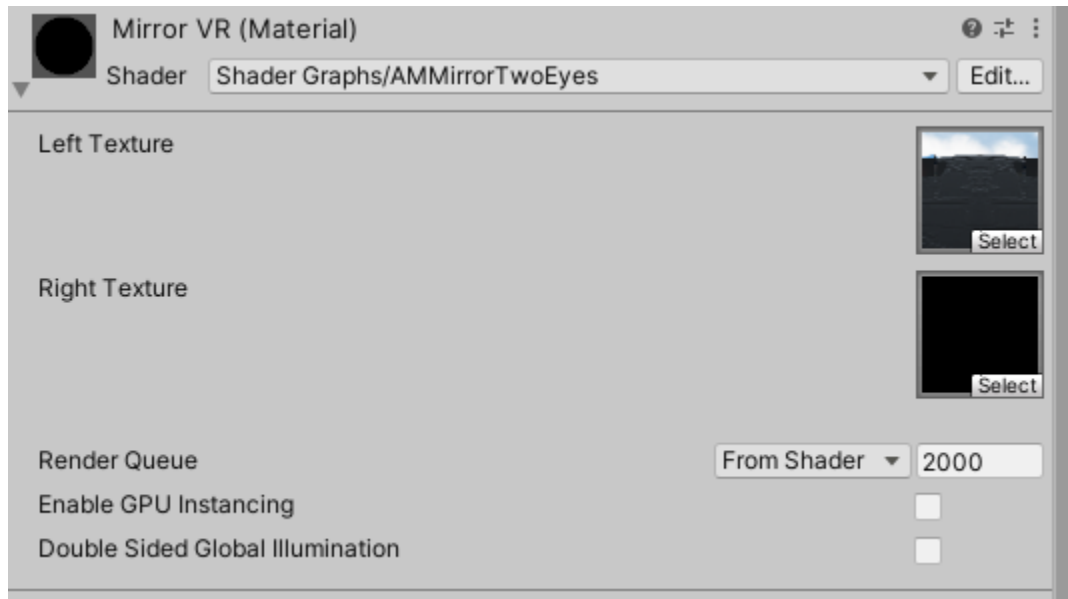


That is all, let's hit the play button :)

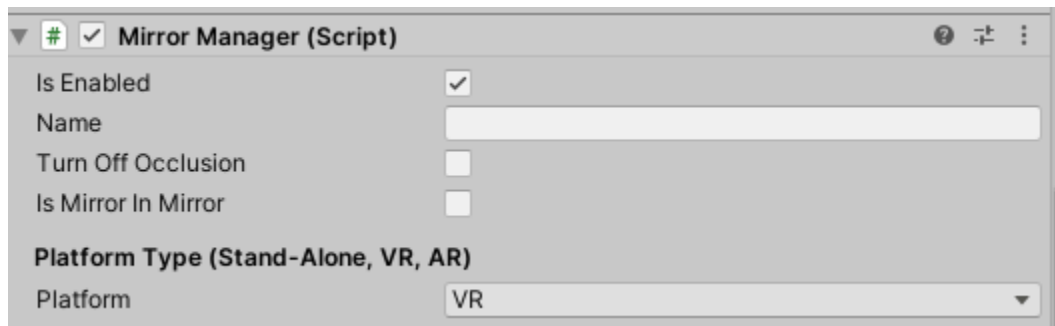


VR Setup:

The VR setup is the same as the setup above. But it uses a “TwoEyes” shader instead of “OneEye” and needs to install the “XR Plugin Management” module from the “Project Settings”.



As you can see you have set your material to “Shader Graphs/AMMirrorTwoEyes” shader. And just select the “Platform” to “VR” on “MirrorManager” script like:

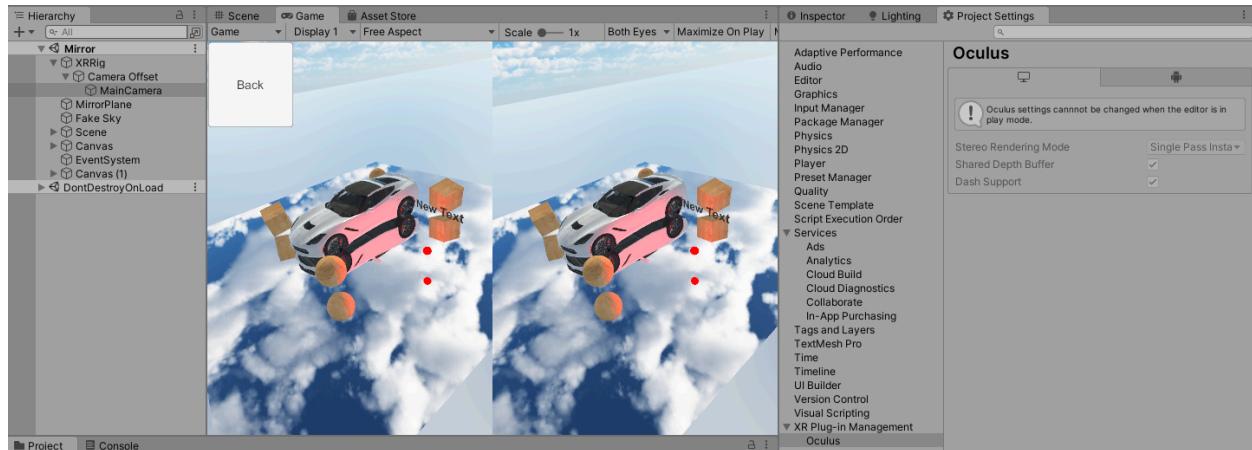


That's all.

I do not go deep into the VR setup because it is related to Unity and you have to know setup and run VR knowledge.

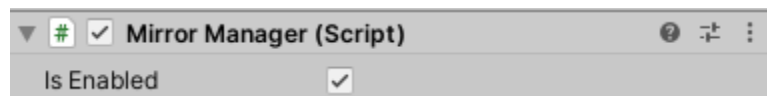
Project includes a “VR Samples” folder as I described above. So you can find the samples there.

Here is a Rift-S screenshot:



Warning: As I said before, you can use the new complex shader for VR too. Please look into the samples folder again.

Mirror Manager Properties:



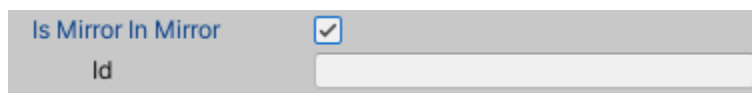
Use this to enable/disable the script (do not use main check on the script itself!). So you can stop-start the reflection as you want.



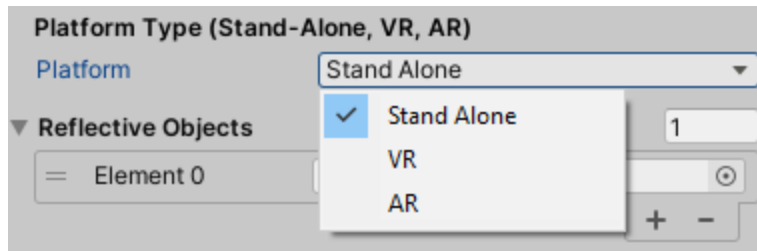
You can name the scripts to understand what they are made for. If you have several mirrors (yes you can create more than one :) Just add another “MirrorManager” to your camera and setup like above for your other mirror) you can forget or can not remember why it is used easily. So it is just a string for you to enter anything.



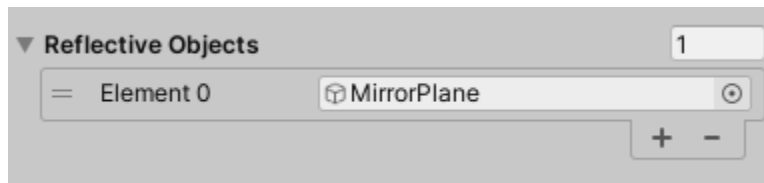
According to camera settings Unity may occlude some objects on mirror render (baked occlusion etc.). So it creates glitches on the reflection (some objects will not be drawn etc.). To disable that you can close the occlusion on the mirror camera.



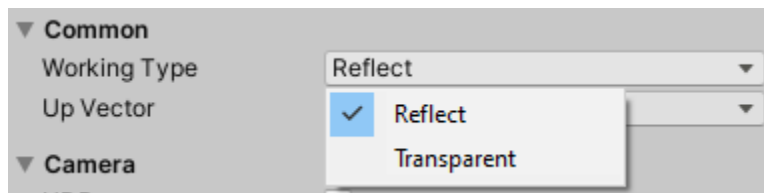
It will be used later to draw recursive (mirror in mirror) mirrors later. This version does not work but you can use previous scripts as I mentioned above. It should be disabled for normal mirrors!



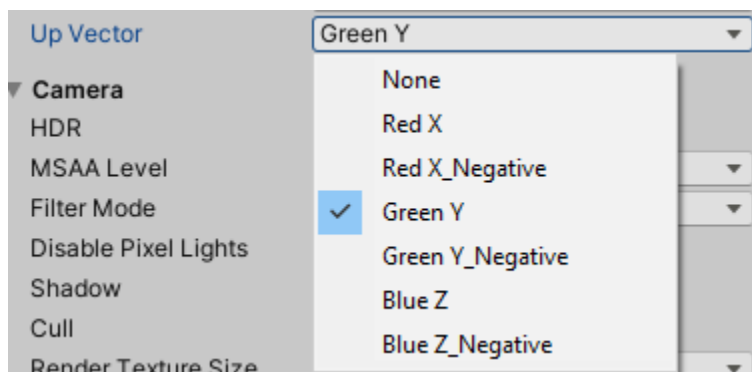
Platform Type: Platform is the main target of the device. So you have to select the correct dropdown item. As you can understand, “Stand Alone” stands for non-VR devices like PC,Mac,Mobile etc. while VR targets VR headsets and AR is special for AR apps.



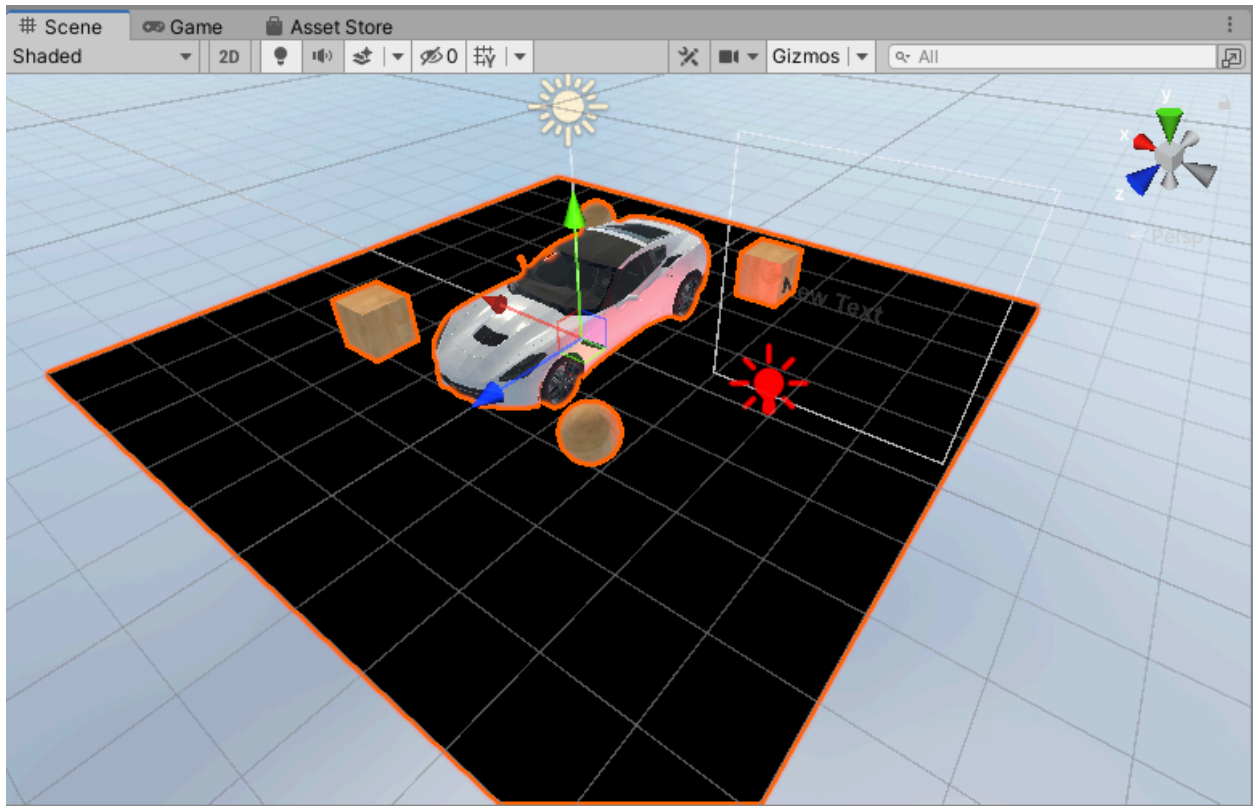
This is the list where mirror objects will be held. Mostly there will be one item (the mirror game object), but if you are using the script for the objects in the same plane direction (like wc mirrors which are all on the same wall), you can just add them here. So the mirror manager will draw the reflection only once and use the same mirror texture on all of them. So you will gain performance.



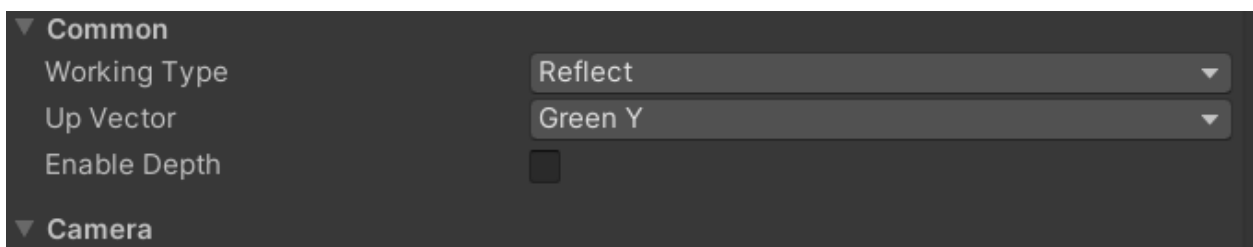
Working Type: This is how the mirror manager draws the reflection. Just now only “Reflect” is active and as you can understand it mirrors the scene. Transparency will be used later for AR and mixed (masked texture blending etc.) cases later.



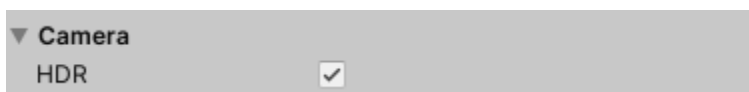
Up Vector is pretty important for the mirror to work. Most of the time the mirror object's facing direction is "Green Y". What that mean is, it is the direction of the object when you select the object on scene view like this:



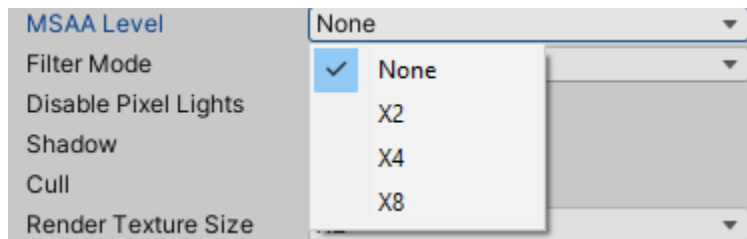
As you can see the floor object's Up Vector (facing the direction where we want the mirroring to happen) is the GreenY vector. But sometimes it can be different (because it can be a sub object in another object, or it may be imported from a 3d modelling program and its vector can be different). So you have to check it and select the correct Up Vector!



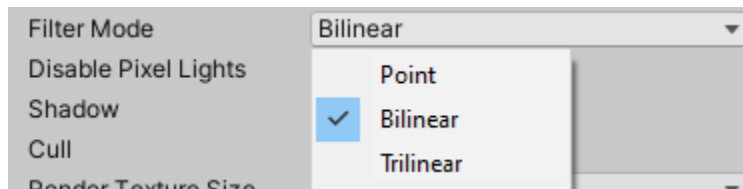
Enable Depth: It enables the depth render to a texture to be used later on shader (material). Look into the "Depth" sample for detail.



Should the camera draw with HDR?



MSAA (Anti aliasing level): It can draw the reflection as anti aliased, but it may decrease the performance specially on mobile platforms. So use it carefully!



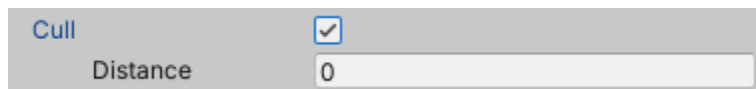
Filter mode is the created mirror texture (render texture) filter mode. So you can select the suitable mode according to your needs.



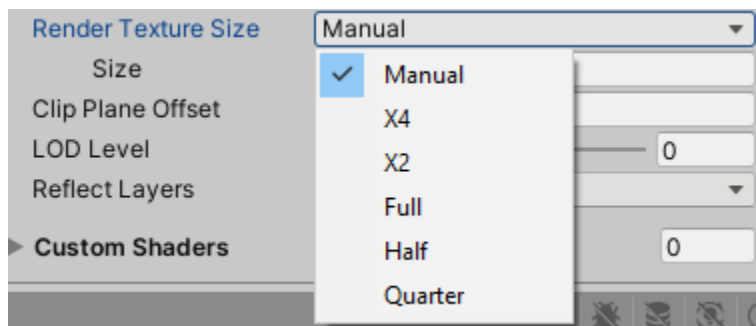
If checked disables the pixel (additional) lights for mirror render. So you can gain performance.



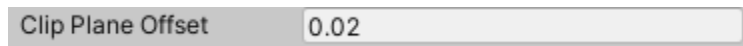
If checked, it enables the shadow on reflection render. But it may create performance costs!



It culls the reflection area in a spherical view. So you can create a max distance to draw the reflection. But it may create not drawn (long distance) areas. So it is good to disable that!



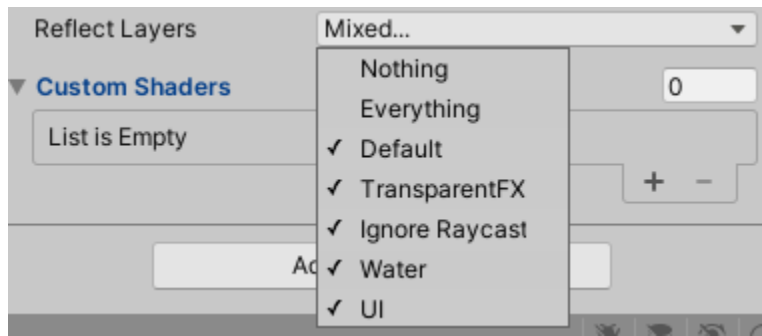
Render texture size is the size of the reflection. For perfect mirrors you should select the “Full” (it creates a screen sized reflection texture to fit exactly on the view). But you can select others or enter the “Manual” mode :)



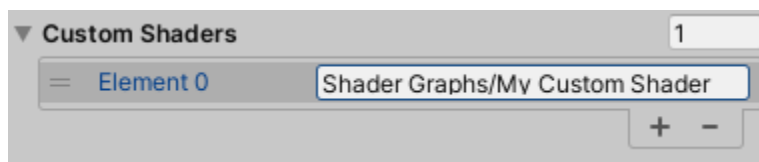
The clipping plane offset of the reflection from the game object. Making this “0” may create a little artifact. So it is good to hold it up a little bit.



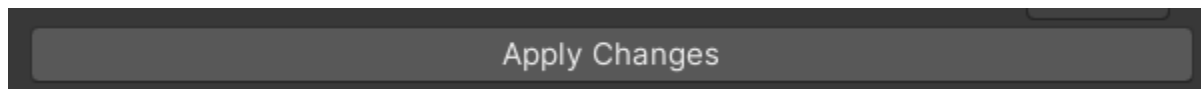
Camera LOD Level to draw begin with. So if you have lods in your scene, you can start to render the reflection from a certain LOD level. So you may gain performance!



You can only reflect certain layers if you want. It is good to reflect everything but you may put your complex objects in a certain layer and exclude it here. So reflection can be drawn faster!



It will be easy to include your custom shader to be used with my reflection textures. This slot will be used for that purpose. Just add your custom shader string path here, and mirror manager will send the textures to your custom shader automatically :) Please look into the “**Reflective Shader Graph**” sample scene for detail!

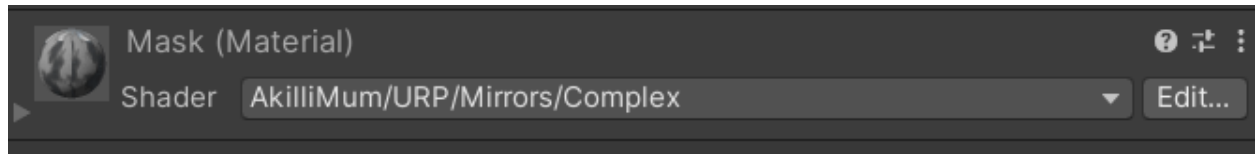


“Apply Changes” button simply applies your script changes! Because to gain speed and some other performance parameters, we do not apply the script changes to the reflection! You have to click this button to see your changes on scene.

New Complex Shader Properties:

Wrote the PBR shader based on URP-Lit shader so you can create your effects more easily:

Sits on:





Shader

AkilliMum/URP/Mirrors/Complex

Edit...

▼ Locally Correction

Enable☐

▼ Reflection

Intensity

1

Disable GI☒

☐ Refraction Map

Depth☐

☐ Alpha Mask Map

▼ Surface Options

Workflow ModeMetallic

Surface TypeOpaque

Render FaceFront

Alpha Clipping☐

Receive Shadows☒

▼ Surface Inputs

☐ Base Map

☐ Metallic Map

0

Smoothness

0

SourceMetallic Alpha

☐ Normal Map

☐ Height Map

☐ Occlusion Map

Clear Coat☐

☐ Mask

0

Smoothness

0

Emission☐

☐ Emission Map

HDR

Tiling

X1Y1

Offset

X0Y0

▼ Detail Inputs

☐ Mask

☐ Base Map

It is based on Lit shader so as you can see starting from “Surface Options” is Lit shader’s properties and we will not go in there :) You know what those properties are and how they work on URP :)



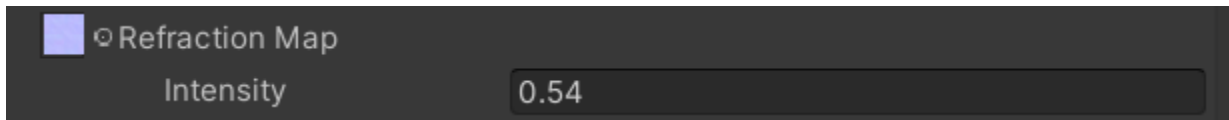
Locally Correction -> Enable: Enables the locally corrected reflection probes on shader. You can use it even without reflection too. So please refer to the associated doc.



Reflection -> Intensity: More intensity, more reflection :) 0 = no reflection, 1 = full mirror



Reflection -> Disable GI: Sometimes even making “Intensity=1” does not create perfect mirrors. Because baked GI (global illumination) can give a little more lighting to material. So checking this removes that artifact. So if you want a perfect mirror, you should check this property:



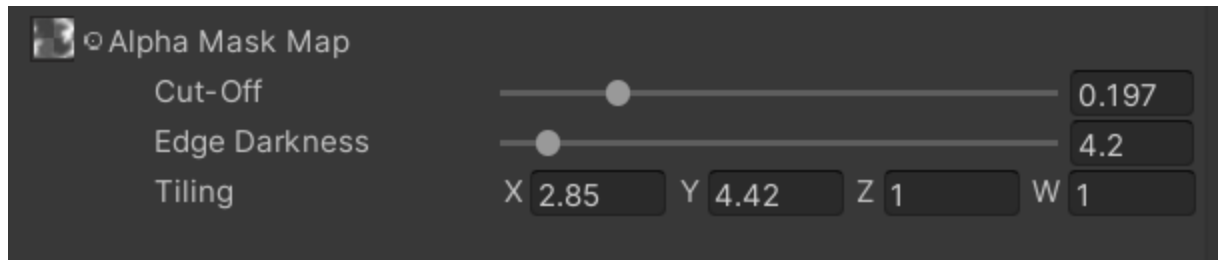
Reflection -> Refraction Map: You can create refracted effects with this normal map on the reflection to mimic ice like surfaces (or floors). **Warning:** the “Surface Inputs->Normal Map” must have a normal map too for this to work. Because it uses the main normal map iteration for calculations!

Reflection -> Refraction Map -> Intensity: More intensity, more refraction :)



Reflection -> Depth: You can create depth based drawing of the reflection on the surface if you enable this (**Warning:** Depth also must be enabled on MirrorManager script).

Reflection -> Depth -> Cut-Off: The cut-off value to set visibility of the depth (reflection) on the surface. Less cut-off, more visibility :)



Reflection -> Alpha Mask Map: You can make only some of the areas of the surface reflective like puddles. So you can use an alpha masked texture here to mimic puddles.

Reflection -> Alpha Mask Map -> Cut-Off: The cut-off value of the mask texture. So you can make the puddles more visible or less.

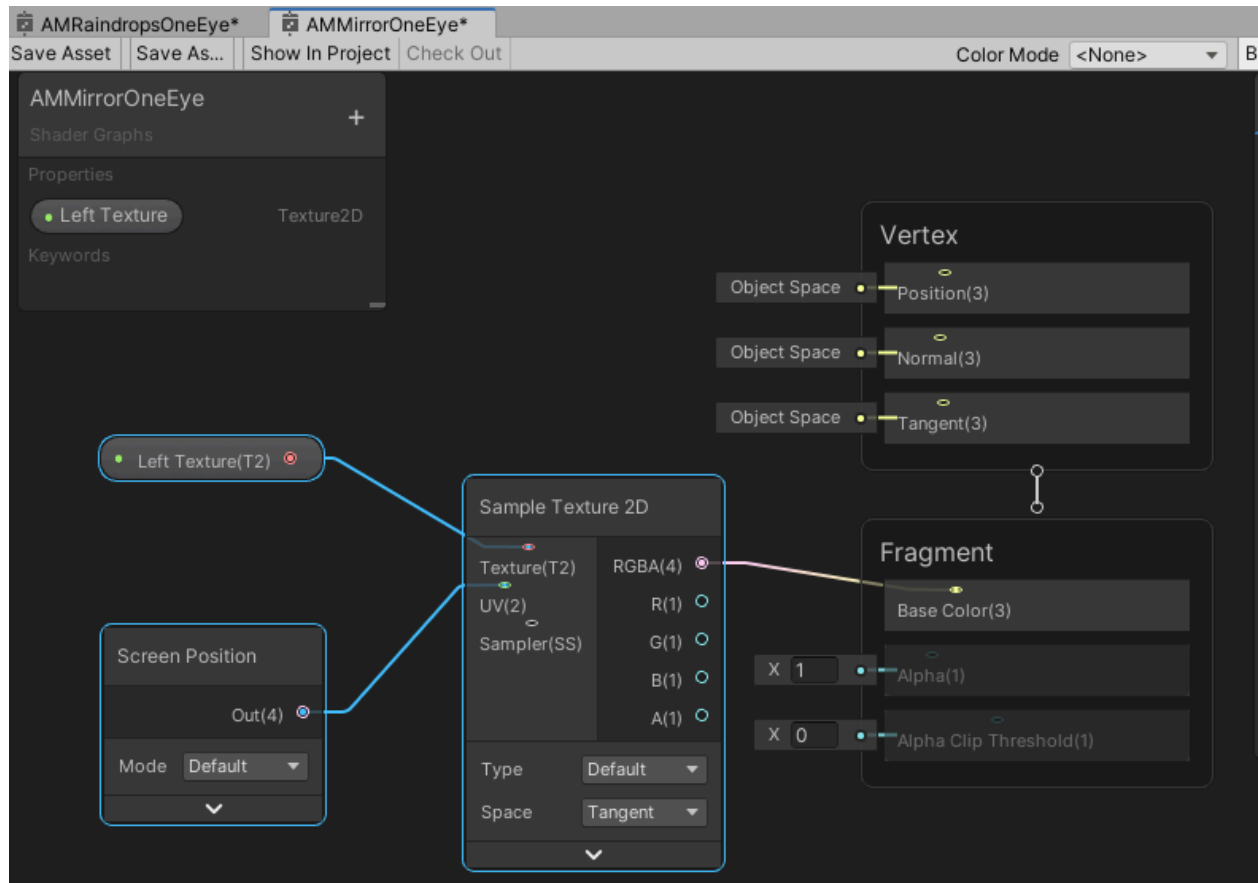
Reflection -> Alpha Mask Map -> Edge Darkness: It is a value to create a darker area on the edges of the mask to mimic some mud like visibility on reflection. Not important but nice in some cases :)

Reflection -> Alpha Mask Map -> Tiling: Just to change tiling of the mask texture on the surface (if it is not created specially for that surface). Only X and Y work to tile it.

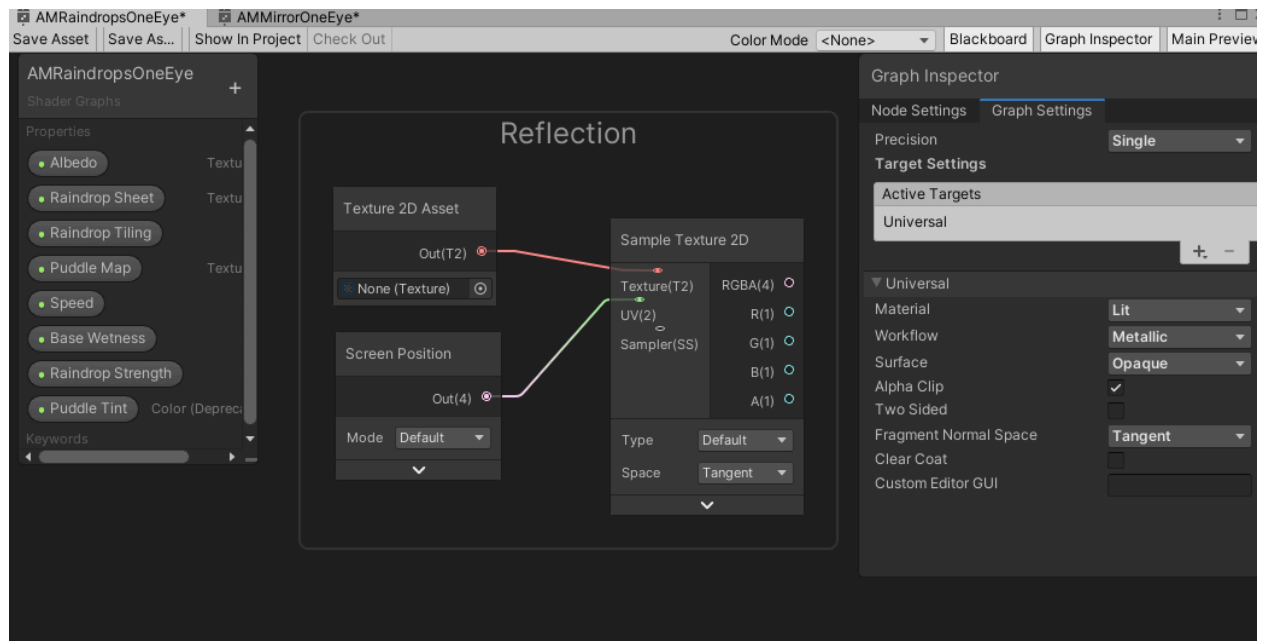
Use on Your Custom Shaders:

You can use the created reflection textures on your custom shader graphs. So you can add reflection to your shaders even on your waters etc. (VR too) Perfect ha? :)

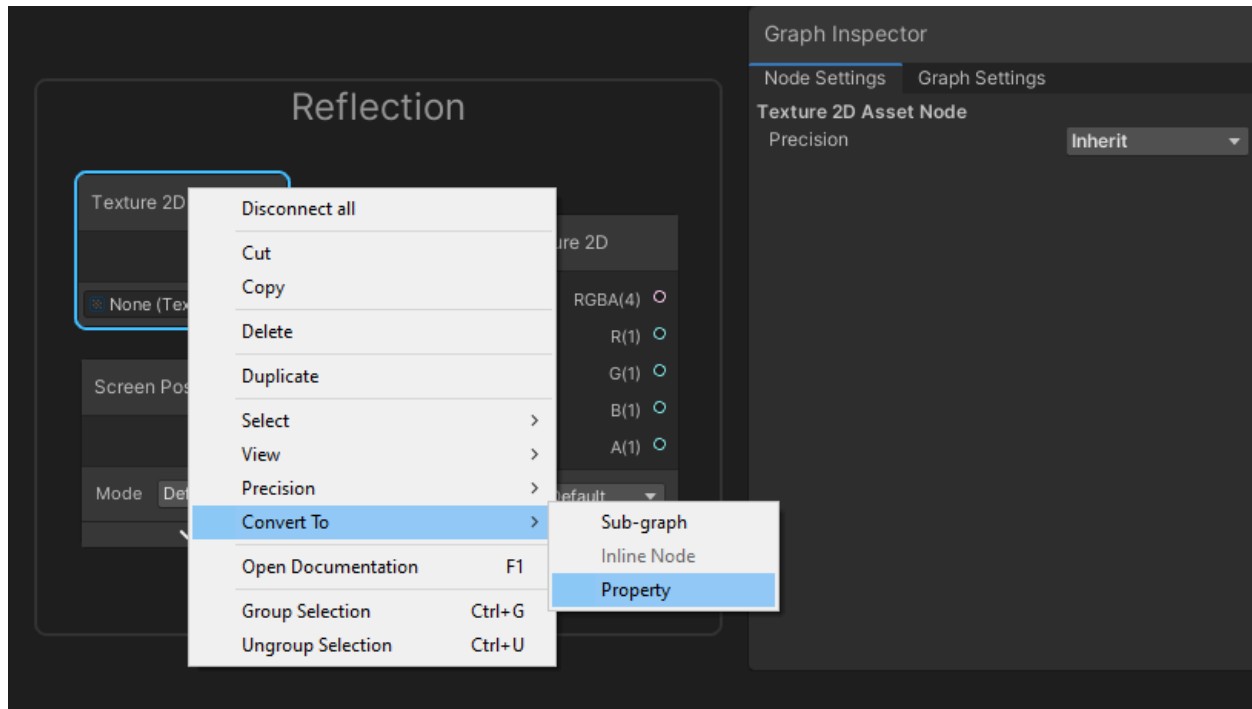
So how to do it? If you open the “AMMirrorOneEye” (or TwoEyes for VR) you will see the whole section simply samples the reflection via screen chords and just pushes it to “Base Color”. So you can copy that nodes:



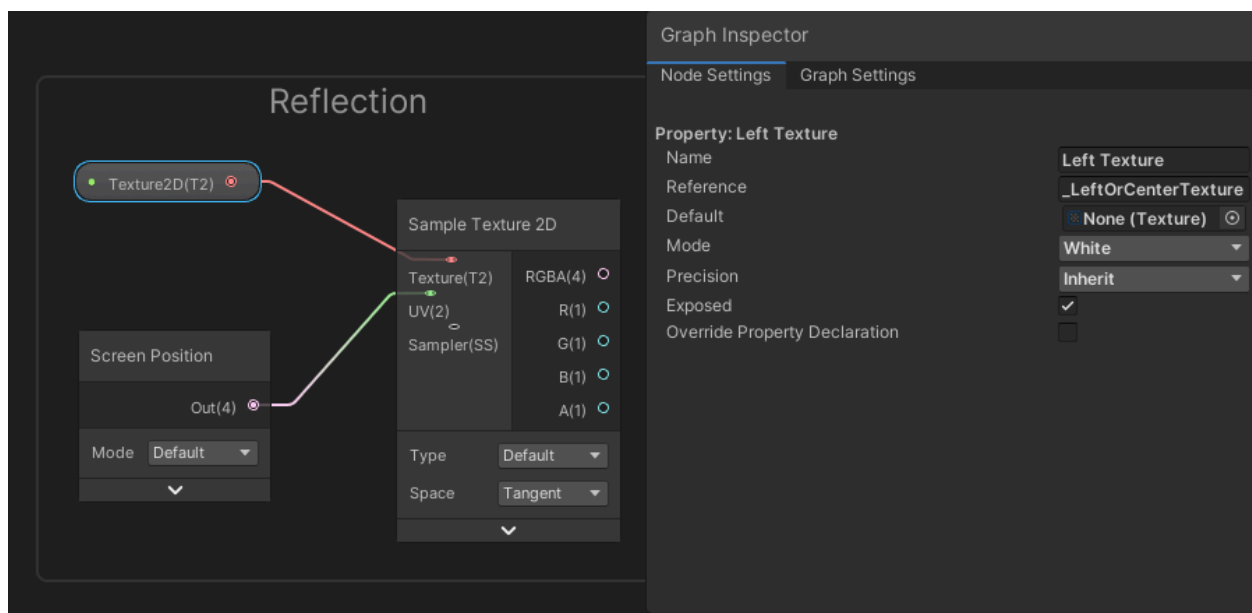
As you can see above for normal apps "OneEye" you can just copy the "LeftTexture" property node, "Screen Position" node and "Sample Texture 2D" node and paste it to your custom shader graph:



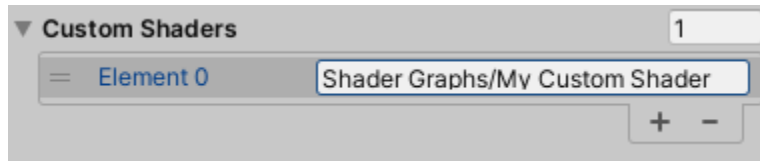
Here I just paste them to another shader graph and just grouped them to make it easy to move. So here “Sample Texture 2D” output (RGBA(4)) is your reflection :) You have to just do one thing! Select the “Texture 2D Asset” (which is our LeftTexture parameter; automatically reverted to its node state on paste) and convert it to “Property” and name as in the previous shader; so it will be like this:



Then change the name and reference to: Name->“LeftTexture” and Reference->“_LeftOrCenterTexture” so it becomes like this:



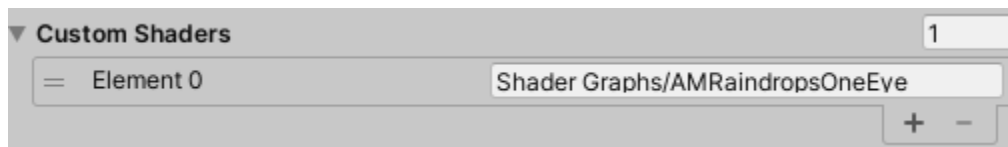
Now your shader is ready to use. You have to just add “MirrorManager” to your camera and set it up as described above. So how will the script find your shader and pass the texture to your shader? If you looked into the script properties you will find a list named as “Custom Shaders”:



So add your shader’s name here as displayed in the material shader selection drop down. For my example it is named as “Shader Graphs/AMRaindropsOneEye”



So i added the name into the “Custom Shaders” slot like this:

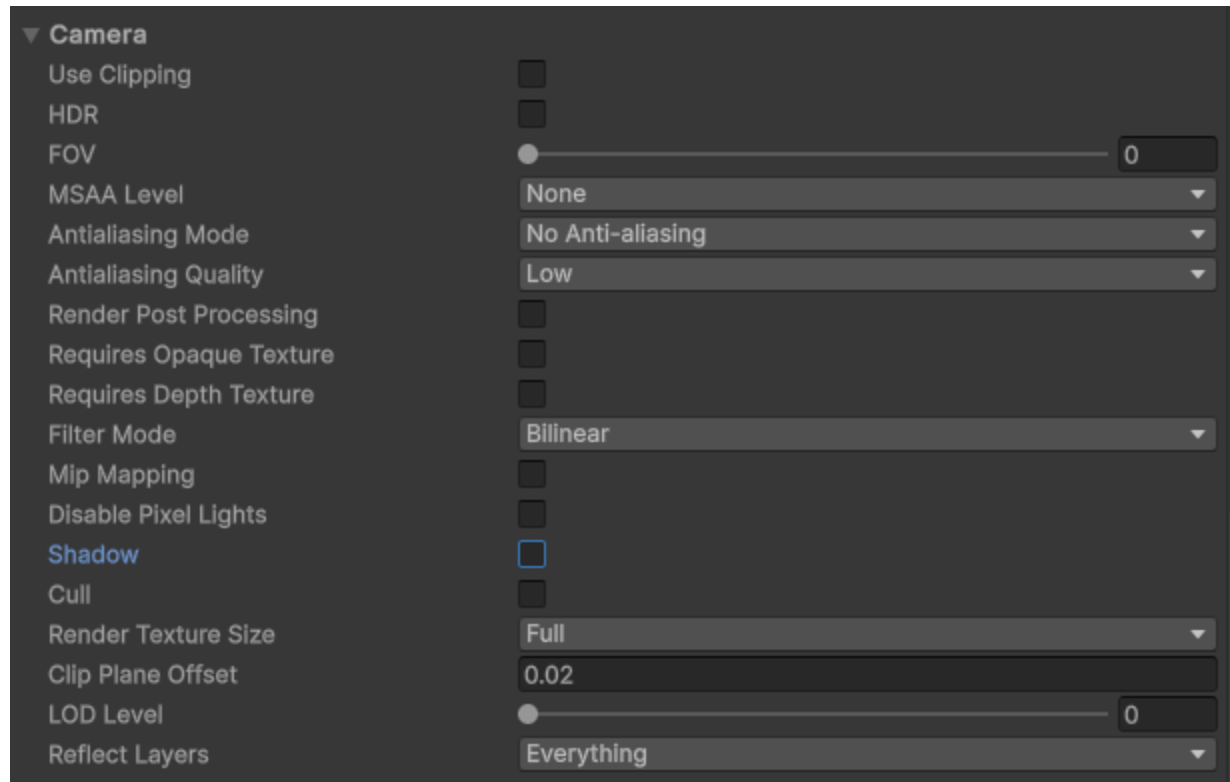


And voile :) You can do the same thing for VR too. The difference is you have to make the right eye texture process like above and copy the “AMMirrorTwoEyes” shader nodes :)

Sample: Please look into the “Reflective” sample scene for a sample of the above process.

Optimization “Mirror Manager Script -> Camera Settings”

Because the script draws the reflection as a second camera, the speed of the game may decrease dramatically. Try below actions to make it reasonable:



HDR: If you enable it, reflection will be drawn with HDR render texture. So it may decrease the speed a little bit.

MSAA Level: Enables the MSAA (Hardware anti aliasing) on reflection. So it may decrease the speed a lot especially on mobile platforms.

Antialiasing Mode: Enables the software anti aliasing mode on reflection camera. It may decrease the speed a little bit.

Render Post Processing: Enables the volume on the reflection camera. So post effects will be enabled on reflection and decrease the speed (like bloom).

Filter Mode: Enables the filtering on the reflection camera. So It may decrease the speed a little bit.

Mip Mapping: Enables the mipmaps on a reflection camera (calculates the mipmaps). Do not open it if you do not need it.

Disable Pixel Lights: As it says :) It does not draw pixel lights (spot and point). So checking it may give you extra speed.

Shadow: Enables the shadows on reflection. Do not check it if you do not really need the reflections.

Cull: It sets the culling distance for the camera to draw. So if you do not need to draw very distance, you can enable it and set a value for the distance.

Render Texture Size: Changes the reflection texture size. Smaller sizes are faster but may become blurry.

LOD Level: To use lower LOD levels to speed up the rendering (of course if you have LOD levels :))

Reflect Layers: Do not render very complex items. Just put them in a different LAYER and unselect that layer here :) So reflection camera will not render that layer (items).

Optimization Samples

Mirror script can drop your frame rate (especially on VR). Because it re-draws the scene from the mirror's object perspective (in VR it draws 2 times, one for each eye). So you have to be careful where to use it and how to use it!

For example:

1:

If you do not want a perfect mirror use the LODs to draw the reflection. For example in my car screens, that is a very detailed car and has 300K triangles. I am using a LOD for the car which has 10K triangles (LOD 1). Then I can select the "Camera LOD Level = 1" from the "CameraShade" script. So it forces the LOD1 (not the real car) for the reflection. So I gain a lot of performance but get smooth semi-reflections!

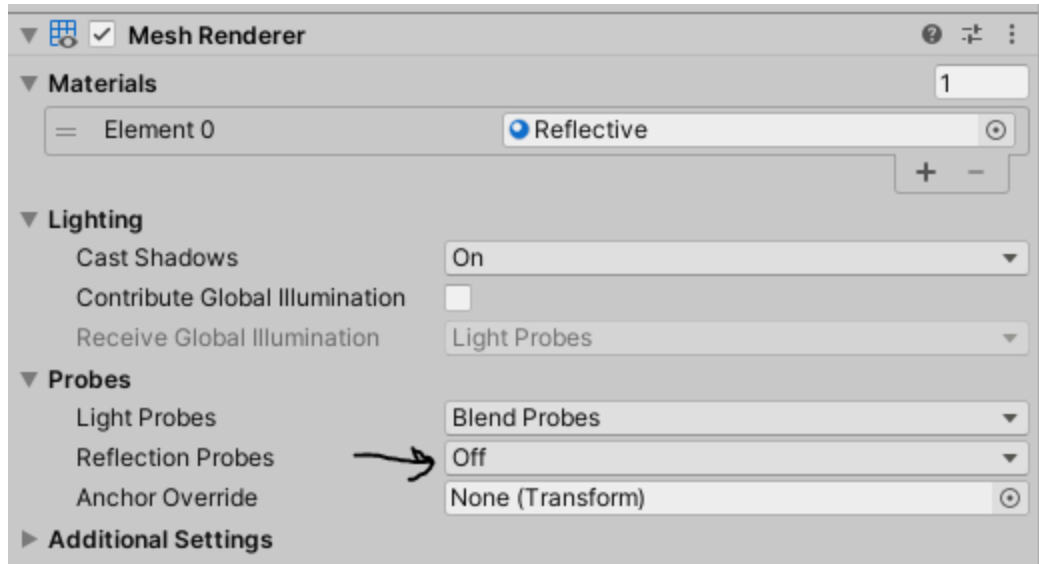
2:

Use different objects and shaders for reflections: You can put your reflection items to different layers and draw only them with the script's "Reflect Layer" option (your main camera also should not draw them again with its layering option). So you can use simplified objects (even with simple shaders) for reflection and gain a lot of performance.

Troubleshoot:

Reflection probes mixes with mirror:

If you increase intensity of the reflection (smoothness), reflection probes may mix with the mirror texture and create unwanted results. So please close the "Reflection Probes" on mesh renderer of the mirror-reflective game object like this (please look into the "Reflective" sample scene as a sample):



Change MirrorManager properties on runtime?

If you look into the MirrorManager script properties you will see the “Apply Changes” button. As we described there, to apply on script changes (to see on reflection) you have to click that button. But it is of course available in the editor :) So what will you do at runtime?

That is pretty simple. MirrorManager holds a public “InitializeMirror” method for that purpose. So after you get the MirrorManager script instance via code like:

```
var manager = gameObject.GetComponent<MirrorManager>(); //etc.
```

```
manager..... //make your changes
```

```
manager.InitializeMirror(); //so you can call like this to re-initialize the script
```

Dynamic Mirrors?

As in the changing MirrorManager properties on runtime (above article), if you are creating mirrors on runtime, you can add the mirror manager script to the scene after you load your mirror, and set up like:

```
var manager = gameObject.AddComponent<MirrorManager>();
```

```
manager.....//add your mirror object to ReflectiveObjects property, setup VR or any other  
//parameters.
```

```
manager.InitializeMirror(); //so you can call like this to re-initialize the script and make it run
```

So you can add the script to your scene and play with its params and reinitialize (like any other script on Unity :)).

Particle system not visible?

As I received some mail, particles may not be visible in some cases. The URP particle system may not write to the Alpha value of the render texture or something related to that. But there seems to be a workaround. Just create a new material and select your particle system shader (as URP->Particles->Unlit etc.) and set the “Render Face=Both”, and voila :)

